MPOO · GRUPO 6 · S18 · 2026-10-19

Encapsular de verdad

Setters, los cuatro modificadores, paquetes, `final`, composición y UML (2.5 y 2.6)

100 minutos · Java 21 · FI-UNAM · 2027-1

Hoy terminamos el pilar del encapsulamiento y aprendemos a dibujar las clases.

EL MAPA DEL CURSO

Hoy completamos el encapsulamiento

Programación orientada a objetos

Abstracción	Encapsulamiento	Herencia	Polimorfismo
Quedarse con lo importante del objeto	Proteger lo de adentro	Reutilizar: un Gerente es un Empleado	Muchas formas, una misma llamada
✔	**HOY, completo**	✔ S16 · P7	S20

Sintaxis de Java: tipos, selección, repetición, arreglos, clases (U1)

En la S16 vimos lo mínimo para poder heredar. Hoy, el encapsulamiento completo.

DE DÓNDE VENIMOS

Lo que ya saben y lo que falta

Ya lo vimos en la S16	Hoy
`private` y getters	Setters que validan
El constructor que valida	Métodos con intención: `aumentar(10)`
`public` y `protected`	El cuarto modificador y los paquetes
«es un» → herencia	«tiene un» → composición, y `final`
Diagramas de clase (S05)	UML con herencia y composición

Cada tema de hoy: primero la idea, luego la tabla, después los ejemplos.

Getter para leer, setter para cambiar

~19 min

Ana recibe un aumento. sueldoBase es private. ¿Cómo lo cambiamos?

SETTERS · LA IDEA

Un setter cambia un atributo privado, si el valor sirve

- El **getter** deja leer: `getSueldoBase()`.
- El **setter** deja cambiar: `setSueldoBase(25000)`.
- El setter **revisa antes**: si el valor no sirve, no cambia nada.
- Mejor aún: un método que dice qué pasa, como `aumentar(10)`.

Sin la revisión, un setter es un public disfrazado.

ANALOGÍA

El cajero automático: no tienes un botón «poner saldo». Tienes depositar y retirar, y cada uno revisa antes.

SETTERS · ¿GETTER Y SETTER PARA TODO?

No: cada atributo se piensa

Atributo	¿Getter?	¿Setter?	Por qué
nombre	Sí	No	No cambia después de crear al empleado (en un momento: `final`)
sueldoBase	Sí	Mejor `aumentar()`	El cambio tiene reglas de negocio
bono	Sí	Sí, con validación	Cambia cada trimestre, pero nunca negativo
cuenta	No	No	Nadie de afuera toca la cuenta (ya lo verán: composición)

IntelliJ genera getters y setters con un clic. Que los genere no quiere decir que todos sirvan.

SETTERS · EL PRIMER INTENTO

Un setter que solo asigna

Empleado.java

```java
class Empleado {
    private String nombre;
    private double sueldoBase;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }

    public double getSueldoBase() { return sueldoBase; }

    public void setSueldoBase(double sueldoBase) {
        this.sueldoBase = sueldoBase;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Ana", 20000);
        e.setSueldoBase(-5000);
        System.out.println(e.getSueldoBase());
    }
}
```

Consola

```
-5000.0
```

Volvimos al sueldo negativo. Un setter sin validación es un public disfrazado.

SETTERS · EL SETTER QUE CUIDA

El setter revisa antes de cambiar

Empleado.java

```java
class Empleado {
    private String nombre;
    private double sueldoBase;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        setSueldoBase(sueldoBase);
    }

    public double getSueldoBase() { return sueldoBase; }

    public void setSueldoBase(double sueldoBase) {
        if (sueldoBase < 0) {
            System.out.println("Rechazado: " + sueldoBase);
            return;
        }
        this.sueldoBase = sueldoBase;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Ana", 20000);
        e.setSueldoBase(-5000);
        System.out.println(e.getSueldoBase());
        e.setSueldoBase(25000);
        System.out.println(e.getSueldoBase());
    }
}
```

Consola

```
Rechazado: -5000.0
20000.0
25000.0
```

Si el valor no sirve, el setter avisa y no toca nada. El sueldo anterior se queda.

SETTERS · UNA SOLA REGLA

El constructor también usa el setter

Empleado.java

```java
class Empleado {
    private String nombre;
    private double sueldoBase;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        setSueldoBase(sueldoBase);
    }

    public double getSueldoBase() { return sueldoBase; }

    public void setSueldoBase(double sueldoBase) {
        if (sueldoBase < 0) {
            System.out.println("Rechazado: " + sueldoBase);
            return;
        }
        this.sueldoBase = sueldoBase;
    }
}
```

Constructor y setter pasan por la misma puerta.

- La regla «no negativo» vive en **un solo lugar**: el setter.
- El constructor no repite el `if`: llama a `setSueldoBase`.
- Si mañana cambia (sueldo mínimo de 8 000), se corrige una línea.
- Si se crea con un valor inválido, el setter también lo rechaza.

✋ **Pregunta · comprobación 1**

¿Qué imprime?

Empleado.java

```java
class Empleado {
    private String nombre;
    private double sueldoBase;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        setSueldoBase(sueldoBase);
    }

    public double getSueldoBase() { return sueldoBase; }

    public void setSueldoBase(double sueldoBase) {
        if (sueldoBase < 0) {
            System.out.println("Rechazado: " + sueldoBase);
            return;
        }
        this.sueldoBase = sueldoBase;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Luis", -100);
        e.setSueldoBase(18000);
        e.setSueldoBase(-1);
        System.out.println(e.getSueldoBase());
    }
}
```

a) `18000.0`

b) `-1.0`

c)
```
Rechazado: -100.0
Rechazado: -1.0
18000.0
```

d) No compila: el constructor recibe un negativo

Fíjense en qué hace el constructor con el -100. Respondan con la letra.

COMPROBACIÓN 1 · RESPUESTA

c) Dos rechazos y se queda en 18000.0

Empleado.java

```java
class Empleado {
    private String nombre;
    private double sueldoBase;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        setSueldoBase(sueldoBase);
    }

    public double getSueldoBase() { return sueldoBase; }

    public void setSueldoBase(double sueldoBase) {
        if (sueldoBase < 0) {
            System.out.println("Rechazado: " + sueldoBase);
            return;
        }
        this.sueldoBase = sueldoBase;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Luis", -100);
        e.setSueldoBase(18000);
        e.setSueldoBase(-1);
        System.out.println(e.getSueldoBase());
    }
}
```

Consola

```
Rechazado: -100.0
Rechazado: -1.0
18000.0
```

	Les faltó ver que...
a	El setter imprime «Rechazado» cada vez
b	Al rechazar, el setter no cambia nada
c	**✔ El constructor también pasa por el setter**
d	javac no revisa valores, solo tipos y acceso

Luis arranca en 0.0 porque el -100 se rechazó, y luego el setter acepta 18000.

SETTERS · MEJOR QUE UN SETTER

Un método que dice qué hace

Empleado.java

```java
class Empleado {
    private String nombre;
    private double sueldoBase;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }

    public double getSueldoBase() { return sueldoBase; }

    public void aumentar(double porcentaje) {
        if (porcentaje <= 0 || porcentaje > 30) {
            System.out.println("No permitido: " + porcentaje);
            return;
        }
        sueldoBase = sueldoBase * (1 + porcentaje / 100);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Ana", 20000);
        e.aumentar(10);
        e.aumentar(80);
        System.out.println(e.getSueldoBase());
    }
}
```

Consola

```
No permitido: 80.0
22000.0
```

setSueldoBase(22000) dice cómo; aumentar(10) dice qué. El objeto pone las reglas del negocio.

Paquetes y los cuatro modificadores

~25 min

Hasta hoy todas sus clases vivían en una sola carpeta. ¿Qué pasa cuando el proyecto crece?

PAQUETES · UN PAQUETE

Un paquete es una carpeta con nombre

Proyecto en IntelliJ

```
S18-Nomina/
└── src/
    ├── nomina/
    │   ├── Empleado.java
    │   └── Gerente.java
    ├── bonos/
    │   └── Vendedor.java
    └── app/
        └── Main.java
```

- Agrupa clases que trabajan juntas.
- Dos clases pueden llamarse igual en paquetes distintos: `nomina.Empleado` y `rh.Empleado`.
- Nombres en minúsculas; la carpeta y el paquete se llaman igual.

> **ANALOGÍA**
> La dirección postal: hay muchas calles Juárez, pero solo una en tu colonia.

Hasta hoy todas sus clases vivían juntas en una sola carpeta: el paquete sin nombre.

PAQUETES · LOS CUATRO NIVELES

Cada nivel abre una puerta más

Modificador	La clase	Mismo paquete	Hija en otro paquete	Cualquiera
`private`	✔	✘	✘	✘
(sin modificador)	✔	✔	✘	✘
`protected`	✔	✔	✔	✘
`public`	✔	✔	✔	✔

ANALOGÍA

Tu casa: tu cuarto es private; la casa entera es el paquete; tu familia, aunque viva en otra casa, es protected; la calle es public.

El renglón amarillo es el nuevo: no escribir nada también es un modificador.

PAQUETES · EN INTELLIJ

Crear un paquete y mover clases

Paso	Qué hacen
1	Clic derecho en `src` → **New → Package** → escriben `nomina`
2	Arrastran `Empleado.java` y `Gerente.java` a la carpeta `nomina`
3	IntelliJ pregunta si mueve la clase (**Refactor**): aceptan
4	IntelliJ escribe solo la línea `package nomina;` y arregla los `import`

No escriban rutas a mano: dejen que IntelliJ mueva y corrija. Luego revisen la primera línea de cada archivo.

PAQUETES · PACKAGE E IMPORT

package dice dónde vive; import dice qué traes

nomina/Empleado.java

```java
package nomina;

public class Empleado {
    private String nombre;
    protected double sueldoBase;

    public Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }

    public String getNombre() { return nombre; }
}
```

nomina/Gerente.java

```java
package nomina;

public class Gerente extends Empleado {
    private double bono;

    public Gerente(String nombre, double sueldoBase, double bono) {
        super(nombre, sueldoBase);
        this.bono = bono;
    }

    public double calcularPago() {
        return sueldoBase + bono;
    }
}
```

app/Main.java

```java
package app;

import nomina.Gerente;

public class Main {
    public static void main(String[] args) {
        Gerente g = new Gerente("Ana", 20000, 5000);
        System.out.println(g.getNombre() + ": " + g.calcularPago());
    }
}
```

Consola

```
Ana: 25000.0
```

package va en la primera línea. Sin el import, Main no sabría qué es Gerente.

PAQUETES · PROTECTED DESDE AFUERA

Main en otro paquete no ve lo protected

nomina/Empleado.java

```java
package nomina;

public class Empleado {
    private String nombre;
    protected double sueldoBase;

    public Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }

    public String getNombre() { return nombre; }
}
```

nomina/Gerente.java

```java
package nomina;

public class Gerente extends Empleado {
    private double bono;

    public Gerente(String nombre, double sueldoBase, double bono) {
        super(nombre, sueldoBase);
        this.bono = bono;
    }

    public double calcularPago() {
        return sueldoBase + bono;
    }
}
```

app/Main.java

```java
package app;

import nomina.Gerente;

public class Main {
    public static void main(String[] args) {
        Gerente g = new Gerente("Ana", 20000, 5000);
        g.sueldoBase = 0;
        System.out.println(g.calcularPago());
    }
}
```

javac

```
app/Main.java:8: error: sueldoBase has
  protected access in Empleado
```

Gerente es hija: sí usa sueldoBase. Main no es hija ni vive en nomina: no entra.

PAQUETES · PROTECTED SÍ SIRVE

La hija en otro paquete sí entra

nomina/Empleado.java

```java
package nomina;

public class Empleado {
    private String nombre;
    protected double sueldoBase;

    public Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }

    public String getNombre() { return nombre; }
}
```

bonos/Vendedor.java

```java
package bonos;

import nomina.Empleado;

public class Vendedor extends Empleado {
    private double ventas;

    public Vendedor(String nombre, double sueldoBase, double ventas) {
        super(nombre, sueldoBase);
        this.ventas = ventas;
    }

    public double calcularPago() {
        return sueldoBase + ventas * 0.05;
    }
}
```

app/Main.java

```java
package app;

import bonos.Vendedor;

public class Main {
    public static void main(String[] args) {
        Vendedor v = new Vendedor("Hugo", 15000, 40000);
        System.out.println(v.getNombre() + ": " + v.calcularPago());
    }
}
```

Consola

```
Hugo: 17000.0
```

Vendedor vive en bonos, pero es hija de Empleado: puede usar sueldoBase. Main no.

PAQUETES · EL OLVIDO MÁS COMÚN

Sin public, la clase no sale de su paquete

nomina/Empleado.java

```java
package nomina;

class Empleado {
    private String nombre;

    public Empleado(String nombre) {
        this.nombre = nombre;
    }
}
```

app/Main.java

```java
package app;

import nomina.Empleado;

public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Ana");
    }
}
```

javac

```
app/Main.java:3: error: Empleado is not public
  in nomina; cannot be accessed from
  outside package
```

Si otro paquete la va a usar, la clase y su constructor llevan public.

✋ **Pregunta · comprobación 2**

Main está en el paquete app. ¿Qué líneas compilan?

`nomina/Empleado.java`

```java
package nomina;

public class Empleado {
    private String nombre;
    double sueldoBase;
    protected double bono;

    public Empleado(String nombre, double sueldoBase, double bono) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        this.bono = bono;
    }

    public String getNombre() { return nombre; }
}
```

`app/Main.java`

```java
package app;

import nomina.Empleado;

public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Ana", 20000, 5000);
        // aquí va una de las cuatro líneas
    }
}
```

a) `System.out.println(e.getNombre());`

b) `System.out.println(e.nombre);`

c) `System.out.println(e.sueldoBase);`

d) `System.out.println(e.bono);`

Puede ser más de una. Busquen cada atributo en la tabla de los cuatro niveles.

COMPROBACIÓN 2 · RESPUESTA

Solo la a

nomina/Empleado.java

```java
package nomina;

public class Empleado {
    private String nombre;
    double sueldoBase;
    protected double bono;

    public Empleado(String nombre, double sueldoBase, double bono) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        this.bono = bono;
    }

    public String getNombre() { return nombre; }
}
```

	¿Compila?	Lo que dice javac
a	**✔ imprime Ana**	**getNombre() es public**
b	✘	nombre has private access in Empleado
c	✘	sueldoBase is not public in Empleado; cannot be accessed from outside package
d	✘	bono has protected access in Empleado (Main no es hija)

La c es la trampa: «sin modificador» no es public, es solo el paquete.

PAQUETES · LA REGLA PRÁCTICA

Qué modificador poner

Qué es	Modificador
Atributos	`private`, siempre
Métodos que otros usan y clases que otros paquetes usan	`public`
Lo que solo las hijas necesitan	`protected`, mejor un getter protected que el atributo
Sin modificador	Casi nunca a propósito: si se les olvidó, quedó de paquete

Empiecen por lo más cerrado y abran solo cuando algo lo necesite.

final: lo que ya no cambia

~8 min

El nombre de un empleado no cambia. ¿Cómo se lo decimos a Java?

FINAL · TRES USOS

final quiere decir «ya no»

Dónde	Quiere decir	Ejemplo
Atributo	Se asigna una vez, en el constructor, y ya no cambia	`private final String nombre;`
Método	Las hijas no lo pueden sobrescribir	`public final String getNombre()`
Clase	Nadie puede heredar de ella	`String` es final

> **ANALOGÍA**
> Tinta indeleble contra lápiz: lo final no se borra; lo demás, sí.

Se usa para lo que no debe cambiar, no para todo.

FINAL · FINAL EN UN ATRIBUTO

Un setter de nombre ya no compila

Empleado.java

```java
class Empleado {
    private final String nombre;
    private double sueldoBase;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

javac

```
Empleado.java:11: error: cannot assign a
  value to final variable nombre
```

El constructor lo asigna una vez. Después, ni la propia clase lo cambia.

FINAL · FINAL EN MÉTODO Y EN CLASE

Ni sobrescribir ni heredar

Empleado.java

```java
class Empleado {
    private final String nombre;

    Empleado(String nombre) {
        this.nombre = nombre;
    }

    public final String getNombre() {
        return nombre;
    }
}
```

Gerente.java

```java
class Gerente extends Empleado {
    Gerente(String nombre) {
        super(nombre);
    }

    @Override
    public String getNombre() {
        return "Lic. " + super.getNombre();
    }
}
```

javac

```
Gerente.java:7: error: getNombre() in
  Gerente cannot override getNombre()
  in Empleado
  overridden method is final
```

MiTexto.java

```java
class MiTexto extends String {
}
```

javac

```
MiTexto.java:1: error: cannot
  inherit from final String
```

@Override y extends tienen un límite, y ese límite es final.

«Tiene un»: composición

~13 min

¿Qué hacemos cuando «es un» suena mal?

COMPOSICIÓN · LA IDEA

Composición: un objeto que tiene a otro

- Una clase guarda como **atributo** un objeto de otra clase.
- Le **delega** trabajo: no lo hace ella, se lo pide al otro.
- Se reconoce con la frase **«tiene un»**; la herencia, con «es un».

Herencia y composición son las dos formas de reutilizar código.

ANALOGÍA

El celular tiene una batería: no es una batería, pero viene con ella y la usa para todo.

COMPOSICIÓN · ES UN O TIENE UN

La prueba de la frase

Frase	¿Suena bien?	Se modela con
Un Gerente **es un** Empleado	Sí	Herencia: `extends`
Un Empleado **es una** CuentaBancaria	No	—
Un Empleado tiene una CuentaBancaria	**Sí**	**Composición: un atributo**
Un Auto **tiene un** Motor	Sí	Composición: un atributo

Cuando «es un» suena raro y «tiene un» suena bien, no es herencia.

COMPOSICIÓN · EN CÓDIGO

Un atributo que es otro objeto

Empleado.java

```java
abstract class Empleado {
    private final String nombre;
    private final double sueldoBase;
    private final CuentaBancaria cuenta;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        this.cuenta = new CuentaBancaria();
    }

    public String getNombre() { return nombre; }
    public double getSueldoBase() { return sueldoBase; }
    public double getSaldo() { return cuenta.getSaldo(); }

    public void cobrar() {
        cuenta.depositar(calcularPago());
    }

    public abstract double calcularPago();
}
```

CuentaBancaria.java

```java
class CuentaBancaria {
    private double saldo;

    public void depositar(double monto) {
        if (monto <= 0) {
            return;
        }
        saldo = saldo + monto;
    }

    public double getSaldo() { return saldo; }
}
```

- La cuenta **nace con** el empleado: `new` en el constructor.
- Es `private final`: nadie de afuera la toca.
- `cobrar()` le **delega** el depósito a su cuenta.

Empleado no sabe depositar: le pide a su cuenta que lo haga.

Ana cobra dos quincenas

Empleado.java

```java
abstract class Empleado {
    private final String nombre;
    private final double sueldoBase;
    private final CuentaBancaria cuenta;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        this.cuenta = new CuentaBancaria();
    }

    public String getNombre() { return nombre; }
    public double getSueldoBase() { return sueldoBase; }
    public double getSaldo() { return cuenta.getSaldo(); }

    public void cobrar() {
        cuenta.depositar(calcularPago());
    }

    public abstract double calcularPago();
}
```

Gerente.java

```java
class Gerente extends Empleado {
    private final double bono;

    Gerente(String nombre, double sueldoBase, double bono) {
        super(nombre, sueldoBase);
        this.bono = bono;
    }

    @Override
    public double calcularPago() {
        return getSueldoBase() + bono;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Gerente ana = new Gerente("Ana", 20000, 5000);
        ana.cobrar();
        ana.cobrar();
        System.out.println(ana.getNombre() + ": " + ana.getSaldo());
    }
}
```

Consola

```
Ana: 50000.0
```

Desde Main nadie puede tocar la cuenta de Ana. Solo cobrar().

COMPOSICIÓN · ¿Y SI HEREDAMOS?

Empleado extends CuentaBancaria compila… y es un error

CuentaBancaria.java

```java
class CuentaBancaria {
    private double saldo;

    public void depositar(double monto) {
        if (monto <= 0) {
            return;
        }
        saldo = saldo + monto;
    }

    public double getSaldo() { return saldo; }
}
```

Empleado.java

```java
class Empleado extends CuentaBancaria {
    private final String nombre;

    Empleado(String nombre) {
        this.nombre = nombre;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado ana = new Empleado("Ana");
        ana.depositar(1000000);
        System.out.println(ana.getSaldo());
    }
}
```

Consola

```
1000000.0
```

Heredar regala todo lo público del padre. Cualquiera le deposita un millón a Ana.

✋ **Pregunta · comprobación 3**

¿Es un o tiene un?

	Par
1	Celular y Batería
2	Paladín y Caballero (su P7)
3	Estación de monitoreo y Sensor
4	SensorTemperatura y Sensor
5	Libro y Página

Escriban en el chat: 1 tiene, 2 es… Lean la frase en voz alta antes de contestar.

COMPROBACIÓN 3 · RESPUESTA

Dos son herencia, tres son composición

	Frase que suena bien	Se modela con
1	Un Celular **tiene una** Batería	Composición
2	**Un Paladín es un Caballero**	**Herencia**
3	Una Estación **tiene** Sensores	Composición (agregación, ya verán)
4	**Un SensorTemperatura es un Sensor**	**Herencia**
5	Un Libro **tiene** Páginas	Composición

La 3 y la 1 se ven igual, pero no son iguales. El diagrama lo distingue.

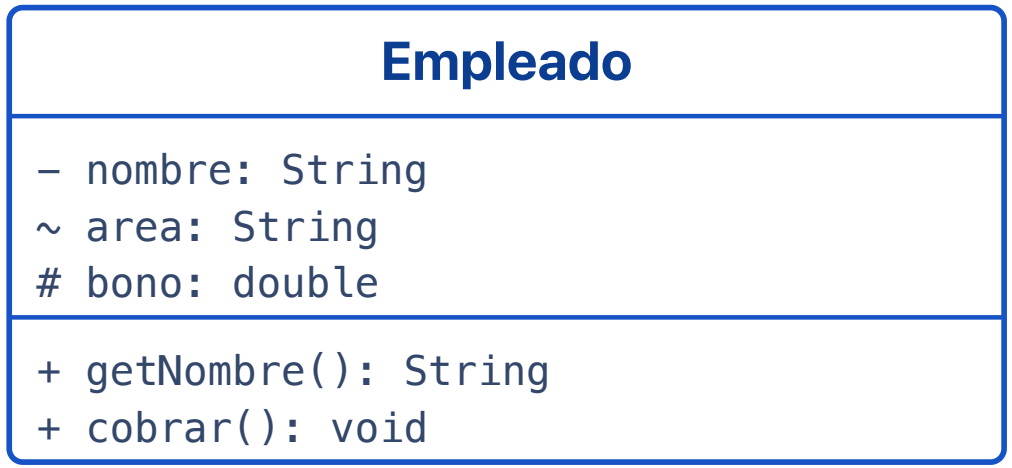
UML: el plano de las clases

~21 min

Todo lo de hoy, dibujado.

UML · LA CAJA (S05)

Tres compartimentos y cuatro símbolos



Símbolo	Modificador
–	private
~	**(sin modificador)**
#	**protected**
+	public

Nombre arriba, atributos en medio, métodos abajo. # y ~ son los que faltaban en la S05.

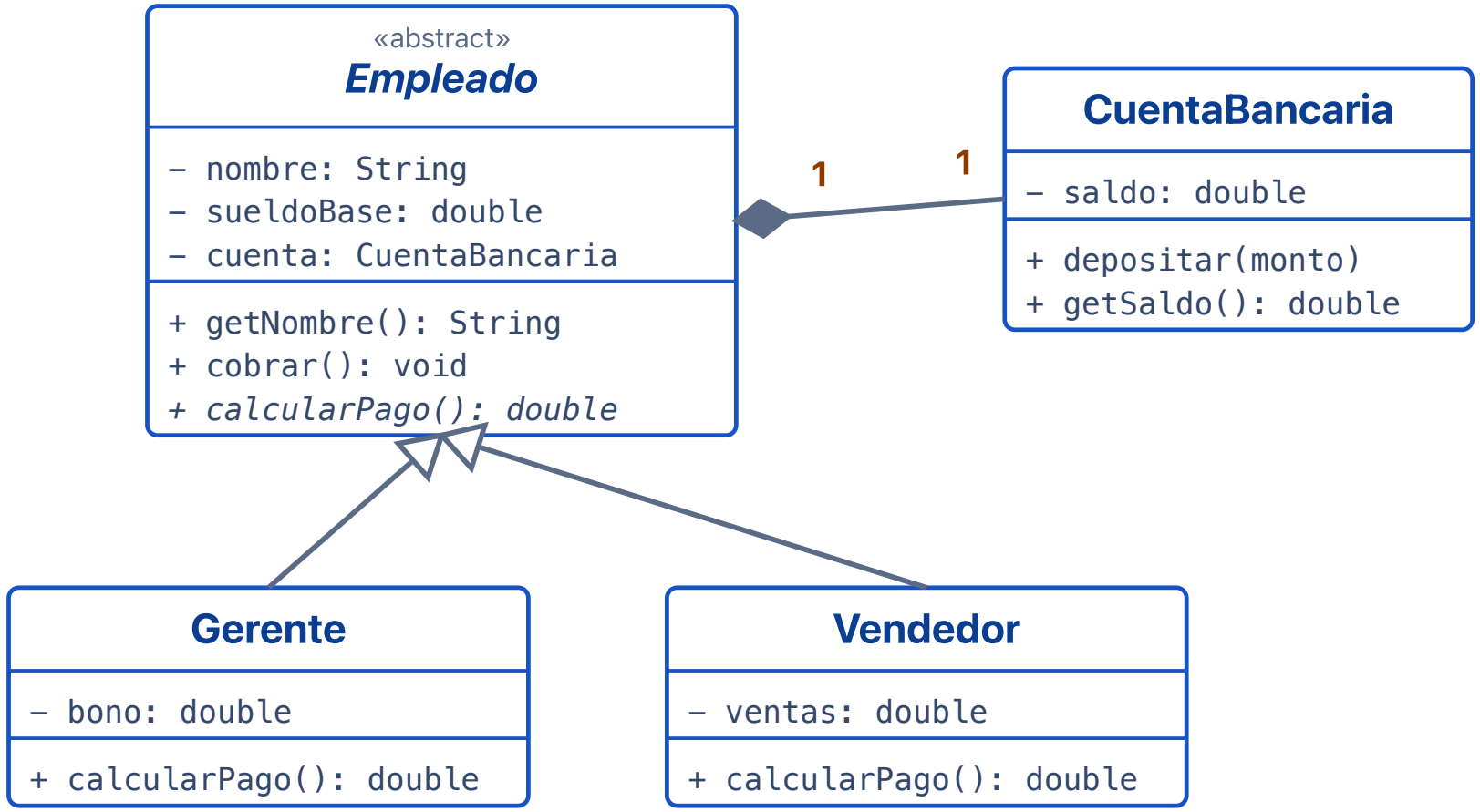
UML · LAS CUATRO LÍNEAS

Cada relación tiene su dibujo y su código

Relación	Dibujo	En Java	Se lee
Generalización	Hija ——▷ Padre	`extends`	es un
Composición	Parte ——◆ Todo	atributo creado con `new` adentro	tiene un, y la parte muere con el todo
Agregación	Parte ——◇ Todo	atributo que llega de afuera	tiene un, pero la parte vive sola
Asociación	A ——— B	uno usa al otro	conoce a

El triángulo y el rombo van del lado del padre y del todo.

Todo lo que tecleamos, en un dibujo



Cursiva = abstract. El rombo lleno: la cuenta nace y muere con el empleado.

UML · COMPOSICIÓN CONTRA AGREGACIÓN

La diferencia está en quién hace el new

Empleado.java

```java
abstract class Empleado {
    private final String nombre;
    private final double sueldoBase;
    private final CuentaBancaria cuenta;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        this.cuenta = new CuentaBancaria();
    }

    public String getNombre() { return nombre; }
    public double getSueldoBase() { return sueldoBase; }
    public double getSaldo() { return cuenta.getSaldo(); }

    public void cobrar() {
        cuenta.depositar(calcularPago());
    }

    public abstract double calcularPago();
}
```

◆ Composición: la crea él

Estacion.java

```java
class Estacion {
    private final String nombre;
    private final Sensor[] sensores;

    Estacion(String nombre, Sensor[] sensores) {
        this.nombre = nombre;
        this.sensores = sensores;
    }

    public void reportar() {
        System.out.println("Estación " + nombre);
        for (Sensor s : sensores) {
            System.out.println(s.getId() + ": " + s.medir() + " (" + s.getCarga() + " %)");
        }
    }
}
```

◇ Agregación: se la pasan

Si desmontan la estación, los sensores se reinstalan en otra. La cuenta de Ana no sobrevive sin Ana.

UML · MULTIPLICIDAD

¿Cuántos con cuántos?

Se escribe	Quiere decir	Ejemplo
`1`	Exactamente uno	Un Empleado tiene 1 CuentaBancaria
`0..1`	Ninguno o uno	Un Empleado tiene 0..1 Gerente a cargo
`*`	Cero o más	Un Gerente tiene * Vendedores
`1..*`	Uno o más	Una Estación tiene 1..* Sensores

Se escribe en cada punta de la línea. En Java, «muchos» casi siempre es un arreglo.

✋ **Pregunta · tu turno · 8 min**

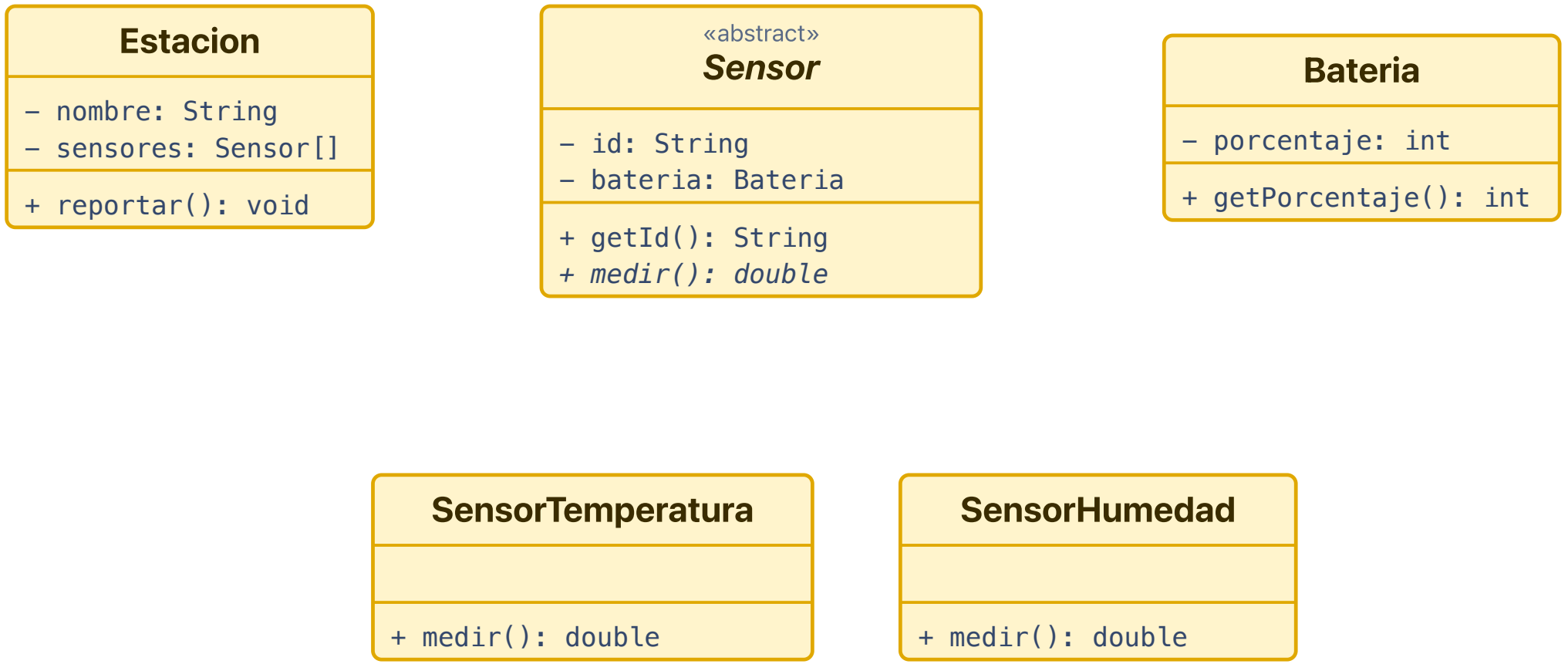
Del enunciado al diagrama

Una **estación de monitoreo** tiene un nombre y uno o más **sensores**. Todo sensor tiene un id, sabe `medir()` y tiene su propia **batería** con un porcentaje de carga. Hay sensores de **temperatura** y de **humedad**, y cada uno mide a su manera. Si la estación se desmonta, los sensores se pueden instalar en otra.

En papel o en draw.io: subrayen los sustantivos (clases) y busquen cada «es un» y cada «tiene un». Foto al chat.

SOLUCIÓN · PASO 1

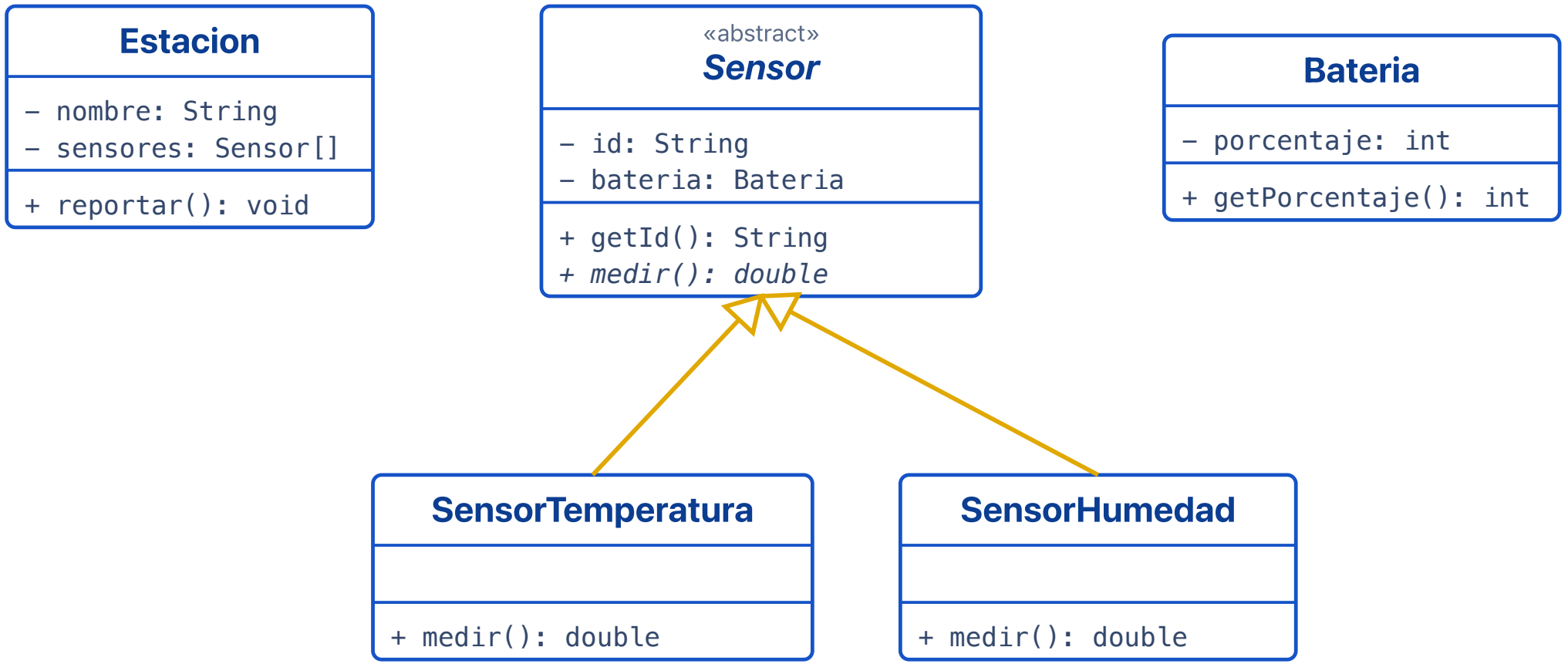
Las clases: los sustantivos del enunciado



Sensor en cursiva: no existe un sensor «a secas», solo de temperatura o de humedad.

SOLUCIÓN · PASO 2

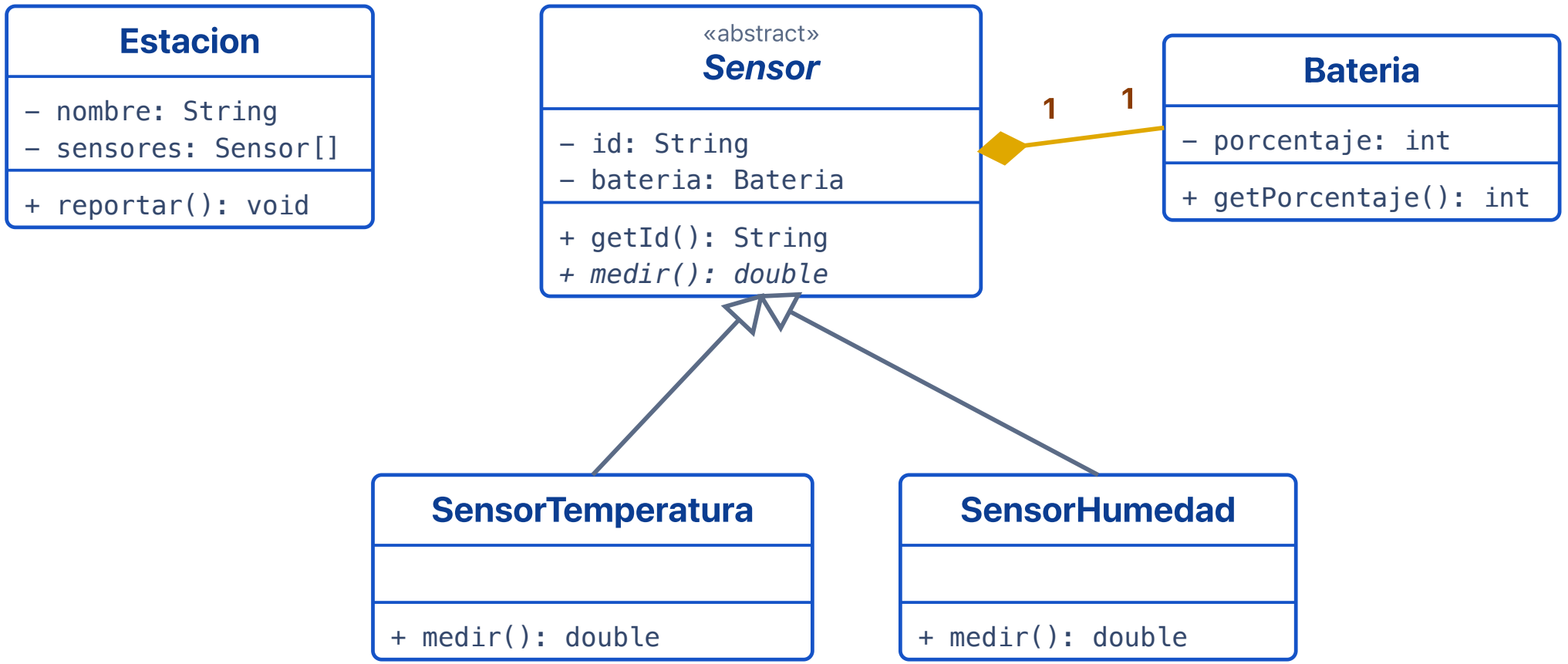
«Es un»: dos generalizaciones



Un SensorTemperatura es un Sensor. Cada hijo escribe su medir().

SOLUCIÓN · PASO 3

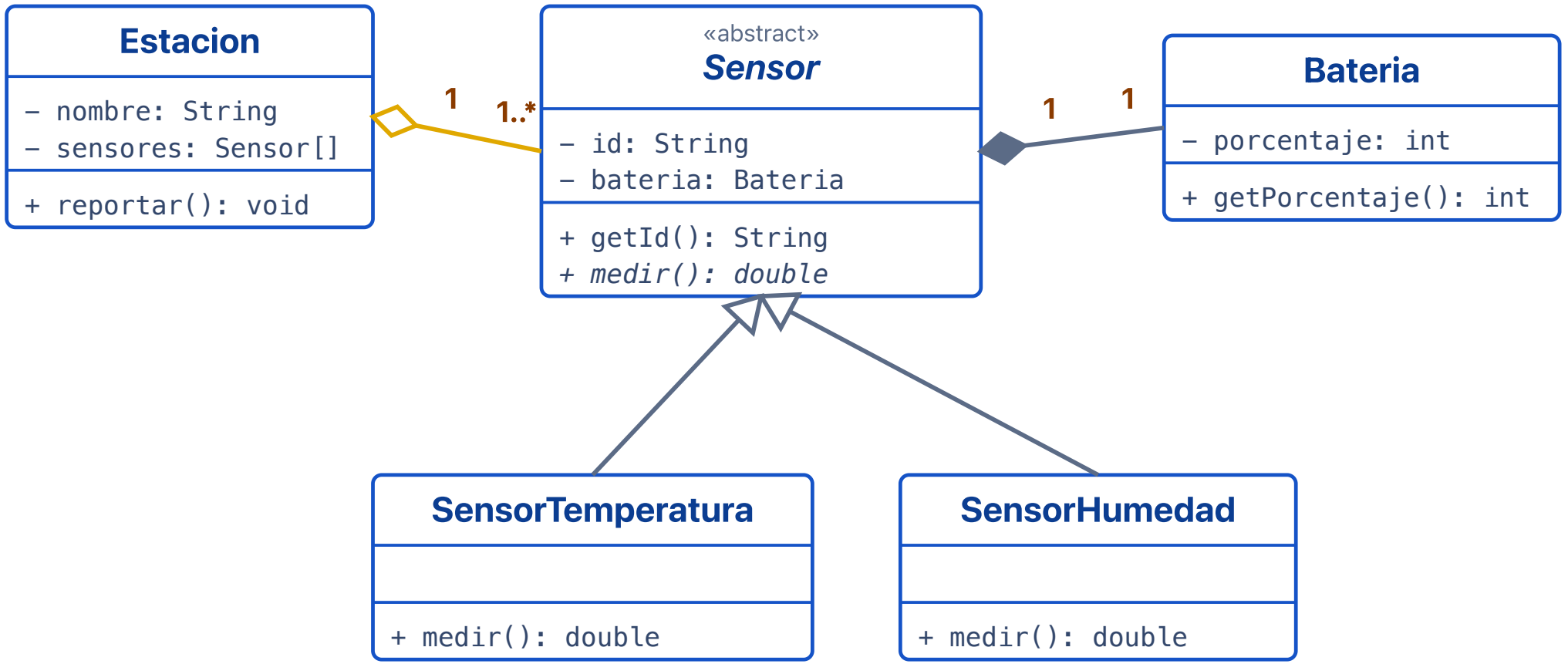
«Tiene una» batería: composición



La batería nace con el sensor (new en su constructor) y es solo suya.

SOLUCIÓN · PASO 4

«Tiene sensores»: agregación, 1..*



Los sensores llegan de afuera y sobreviven a la estación: rombo hueco.

SOLUCIÓN · EN JAVA

La jerarquía y la batería

Sensor.java

```java
abstract class Sensor {
    private final String id;
    private final Bateria bateria;

    Sensor(String id) {
        this.id = id;
        this.bateria = new Bateria();
    }

    public String getId() { return id; }
    public int getCarga() { return bateria.getPorcentaje(); }

    public abstract double medir();
}
```

Bateria.java

```java
class Bateria {
    private int porcentaje = 100;

    public int getPorcentaje() { return porcentaje; }
}
```

SensorTemperatura.java

```java
class SensorTemperatura extends Sensor {
    SensorTemperatura(String id) { super(id); }

    @Override
    public double medir() { return 21.5; }
}
```

SensorHumedad.java

```java
class SensorHumedad extends Sensor {
    SensorHumedad(String id) { super(id); }

    @Override
    public double medir() { return 40.0; }
}
```

Generalización = extends; composición = new Bateria() dentro del constructor de Sensor.

SOLUCIÓN · EN JAVA

La estación y su main, y corre

Estacion.java

```java
class Estacion {
    private final String nombre;
    private final Sensor[] sensores;

    Estacion(String nombre, Sensor[] sensores) {
        this.nombre = nombre;
        this.sensores = sensores;
    }

    public void reportar() {
        System.out.println("Estación " + nombre);
        for (Sensor s : sensores) {
            System.out.println(s.getId() + ": " + s.medir() + " (" + s.getCarga() + " %)");
        }
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Sensor[] sensores = {
            new SensorTemperatura("T1"),
            new SensorHumedad("H1")
        };
        Estacion azotea = new Estacion("Azotea", sensores);
        azotea.reportar();
    }
}
```

Consola

```
Estación Azotea
T1: 21.5 (100 %)
H1: 40.0 (100 %)
```

Agregación = los sensores llegan por el constructor de Estacion: se crean afuera y sobreviven a ella.

Cierre

~8 min

Qué nos llevamos y qué sigue el miércoles.

RESUMEN

Cinco ideas de hoy

Idea	En una línea
Setter que valida	Cambiar sí, pero el objeto revisa; mejor aún, un método con intención
Cuatro modificadores	private, (paquete), protected, public: cada uno abre una puerta más
package e import	La carpeta donde vive la clase y lo que traes de otra
final	Ya no cambia, ya no se sobrescribe, ya no se hereda
Composición y UML	«Tiene un» es un atributo; ◆ si lo crea él, ◇ si se lo pasan

Si pueden explicar cada fila con la nómina, entendieron la clase.

✋ **Pregunta · salida**

La batería no puede pasar de 100 %

Escribe en `Bateria` los métodos `cargar(int cuanto)` y `gastar(int cuanto)`.
Nunca debe quedar arriba de 100 ni abajo de 0, y no se vale pasar un número negativo.

Es el setter que valida de hoy, con dos métodos con intención. En el chat o en su cuaderno.

PREGUNTA DE SALIDA · RESPUESTA

Dos métodos que cuidan los límites

Bateria.java

```java
class Bateria {
    private int porcentaje = 100;

    public int getPorcentaje() { return porcentaje; }

    public void cargar(int cuanto) {
        if (cuanto <= 0) {
            return;
        }
        porcentaje = Math.min(100, porcentaje + cuanto);
    }

    public void gastar(int cuanto) {
        if (cuanto <= 0) {
            return;
        }
        porcentaje = Math.max(0, porcentaje - cuanto);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Bateria b = new Bateria();
        b.gastar(70);
        b.cargar(50);
        System.out.println(b.getPorcentaje());
        b.cargar(-500);
        System.out.println(b.getPorcentaje());
    }
}
```

Consola

```
80
80
```

Sin setPorcentaje: nadie de afuera puede poner 150.

MIÉRCOLES 21 · Q007

P8 Herencia, segunda parte

- Extienden su jerarquía con clases abstractas y **composición**, y justifican dónde compusieron en vez de heredar.
- Antes de la clase: la **corrección del parcial a mano**, escaneada en su rama.
- Y la tarea de **colecciones** (ArrayList, HashMap, HashSet) con un ejercicio propio.

Lo de hoy es lo que van a teclear y dibujar el miércoles.